

СТРУКТУРЫ ДАННЫХ ИЗНУТРИ • ДЕНЬ 4

# Строки и структуры

---

*Почему `sys.getsizeof(1)` равно 28, и почему строки  
нельзя изменять?*

## Вопрос дня

---

```
>>> import sys
>>> sys.getsizeof(1)
28
>>> sys.getsizeof(2 ** 30)
32
>>> sys.getsizeof("")
41
>>> s = "hello"
>>> s[0] = "H"
TypeError: 'str' object does not
support item assignment
```

Единица занимает 28 байт.  
Само число помещается в  
четыре, остальные 24 заняты  
чем-то ещё.

## Вопрос дня

---

```
>>> import sys
>>> sys.getsizeof(1)
28
>>> sys.getsizeof(2 ** 30)
32
>>> sys.getsizeof("")
41
>>> s = "hello"
>>> s[0] = "H"
TypeError: 'str' object does not support item assignment
```

Единица занимает 28 байт. Само число помещается в четыре, остальные 24 заняты чем-то ещё.

Пустая строка занимает 41 байт, и в ней нет ни одного символа. Строка из одного символа занимает 42 байта.

Сегодня разбирается, что лежит в этих байтах, почему запись в строку запрещена и что запрет даёт взамен.

РАЗДЕЛ 1

# Строки

---

## Символ — это число

```
char c = 'A';  
printf("%c %d\n", c, c);      // A 65  
printf("%d\n", 'a' - 'A');    // 32  
printf("%d\n", '7' - '0');    // 7
```

Тип `char` — целый тип шириной один байт. На учебной машине он знаковый, и его диапазон — от  $-128$  до  $127$ . Одиночные кавычки дают код символа, и с ним можно делать всё, что с числом.

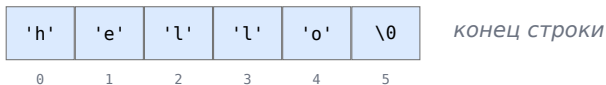
### Ловушка: `sizeof('A')` равен 4

Литерал `'A'` в C имеет тип `int`, а не `char`. Переменная `c` занимает один байт, литерал — четыре.

В Python отдельного типа для символа нет: `'A'` — строка длины 1, а код буквы даёт `ord('A')`.

## Строка — это массив, в конце которого стоит ноль

```
char s[] = "hello";
```



Ячеек шесть, букв пять. Последний байт — ноль, и он не изображается при печати. Отдельного типа «строка» в С нет: есть массив `char` и договорённость, что строка кончается там, где встретился нулевой байт.

### Главное

Все библиотечные функции ищут конец строки перебором байтов. Строка без завершающего нуля приведет к выходу за границу массива.

## sizeof и strlen отвечают на разные вопросы

---

```
char s[] = "hello";  
printf("%zu\n", sizeof s);      // 6 – сколько ячеек в массиве  
printf("%zu\n", strlen(s));    // 5 – сколько байт до нуля
```

`sizeof` считает компилятор, и он знает тип массива. `strlen` считает программа во время работы, проходя по байтам.

Ноль, записанный в середину, укорачивает строку, не трогая массив:

```
s[3] = '\\0';  
printf("%s %zu %zu\n", s, strlen(s), sizeof s); // Hel 3 6
```

Буквы `'l'` и `'o'` остались на местах, но ни одна строковая функция до них уже не доберётся.

# Символ и байт

```
strlen("hello")    // 5
strlen("привет")   // 12
strlen("ёж")        // 4
```

```
>>> len("привет")
6
>>> len("привет".encode())
12
```

Русская буква записывается в UTF-8 двумя байтами, и `strlen` считает именно байты. `s[0]` даёт первый **байт** — половину буквы, у которой нет собственного изображения.

## Главное

Питоновская строка — последовательность символов, *сишная* — последовательность байтов. Ближайший родственник *сишной* строки в Python называется `bytes`: его и возвращает `encode`. Пока в задачах символ размером с байт, разница не проявляется, что может привести к тихой ошибке.

## Что создаёт запись в кавычках

Выражение `"hello"` создаёт безымянный массив `char` из шести элементов: пять букв и завершающий ноль. Массив живёт до конца программы, а его имя распадается в указатель на первый элемент — как у массивов вчера.

| выражение                   | значение              | почему                           |
|-----------------------------|-----------------------|----------------------------------|
| <code>sizeof "hello"</code> | 6                     | это массив, а не указатель       |
| <code>sizeof ""</code>      | 1                     | один байт, и в нём ноль          |
| <code>"hello"[1]</code>     | <code>'e'</code>      | к массиву обращаются по индексу  |
| <code>sizeof "a\0b"</code>  | 4                     | <code>strlen</code> здесь даёт 1 |
| <code>"abc" "def"</code>    | <code>"abcdef"</code> | соседние литералы склеиваются    |

### Главное

`'a'` — число 97 размером с `int`, `"a"` — массив из двух байт. Кавычки определяют тип.

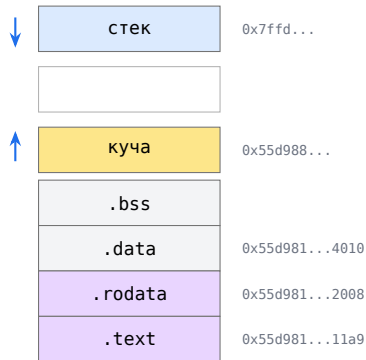
## Два способа записать одно и то же

```
char    array[] = "hello";    // массив на стеке, заполнен копией
const char *pointer = "hello"; // указатель на сам литерал
```

|                   | char array[]     | const char *pointer |
|-------------------|------------------|---------------------|
| что за переменная | массив из 6 байт | указатель, 8 байт   |
| sizeof            | 6                | 8                   |
| где лежат буквы   | в кадре функции  | в .rodata           |
| изменение         | законно          | <b>UB</b>           |
| живёт             | до конца вызова  | до конца программы  |

Литерал существует в единственном экземпляре: два одинаковых литерала в программе — обычно один и тот же адрес. Присваивание `array[0] = 'H'` меняет конкретную копию, как и ожидается от массива.

# Области памяти процесса



- `.text` — машинный код, только чтение.
- `.rodata` — литералы и `const`-данные, только чтение.
- `.data` — глобальные и `static` с начальным значением.
- `.bss` — то же без начального значения, обнуляется при запуске.
- **куча** — `malloc`, растёт вверх.
- **стек** — кадры вызовов, растёт вниз.

Первые четыре берутся из файла программы и живут до конца работы. Адреса фиксированны для конкретного запуска: при следующем они будут другими.

## Что из этого следует

```
const char *literal = "hello"; // указатель и буквы — в разных областях
```

Переменная `literal` лежит по адресу `0x55d981...4018`, в `.data`, а строка, на которую она показывает, — по `0x55d981...2008`, в `.rodata`. Вчерашняя разница между `p` и `&p` — дополнительно разница между областями.

### `.bss` ничего не стоит в файле

`int big[1000000];` увеличивает программу на 24 байта,  
`int big[1000000] = {1};` — на четыре мегабайта: в файле записано либо число нулей, либо сами нули.

Размеры трёх областей видны ещё до запуска:

```
$ size areas
   text    data     bss      dec     hex  filename
   2420     640      16    3076    c04   areas
```

## Запись в литерал

```
char *literal = "hello";  
literal[0] = 'H';
```

### Неопределённое поведение

Segmentation fault (core dumped)

Область `.rodata` помечена при загрузке как доступная только для чтения, и запись в неё останавливает процесс. Компилятор при этом не сказал ничего: тип `char *` записи не запрещает.

### Пишите const

Объявление `const char *s` превращает ту же ошибку в отказ компилятора. В Python она обнаруживается ещё раньше и называется яснее: `TypeError` вместо падения по сигналу.

## Кавычки в инициализаторе — особый случай

```
const char *p = "hello"; // указатель на объект в .rodata
char s[] = "hello"; // массив s из этих букв
```

В первой строке объект есть, и его видно в файле программы. Во второй кавычки задают начальное значение массива, и отдельного объекта может не быть вовсе:

```
$ strings prog | grep hello # с указателем: hello, с массивом: пусто
```

Буквы попали прямо в код, двумя командами процессора:

```
movl $0x6c6c6568, -0xe(%rbp) # h e l l
movw $0x6f, -0xa(%rbp) # o и ноль
```

### Литерал можно вернуть из функции

`return "hello";` законно, ХОТЯ `return local_array;` вчера давало висячий указатель: литерал лежит не в кадре вызова.

## Что есть в string.h

| функция                       | что делает          | в Python                      |
|-------------------------------|---------------------|-------------------------------|
| <code>strlen(s)</code>        | длина до нуля       | <code>len(s)</code>           |
| <code>strcmp(a, b)</code>     | сравнение по кодам  | <code>a == b, a &lt; b</code> |
| <code>strcpy(dst, src)</code> | копирование с нулём | <code>b = a</code>            |
| <code>strcat(dst, src)</code> | дописать в конец    | <code>a += b</code>           |
| <code>strchr(s, c)</code>     | адрес первого c     | <code>s.find(c)</code>        |
| <code>strstr(s, sub)</code>   | адрес подстроки     | <code>sub in s</code>         |

`strcmp` возвращает ноль при совпадении, отрицательное число, если первая строка меньше, и положительное, если больше. Сравниваются коды байтов, поэтому `"Zoo"` меньше `"abc"`.

Оператор `==` для двух `char *` сравнивает адреса, а не текст: он соответствует питоновскому `is`. Путаница между `==` и `strcmp` — неприятная молчаливая ошибка в работе со строками.

## Чего эти функции не знают

```
char buffer[8];  
strcpy(buffer, "Konstantinopolsky"); // 18 байт в 8 ячеек
```

`strcpy` получает два адреса и ни одного размера. Она копирует, пока не встретит ноль в источнике.

### Неопределённое поведение

```
ERROR: AddressSanitizer: stack-buffer-overflow  
WRITE of size 18 at 0x7b0ab11f0048 thread T0
```

Размер назначения принимает `snprintf`, и он же обрезает лишнее:

```
snprintf(buffer, sizeof buffer, "%s", incoming);
```

Результат гарантированно успешен, а возвращает `snprintf` длину, которой **нехватило** для полного копирования: по ней видно, что строку обрезали.

# Строка в куче: не забудьте байт

```
char *my_strdup(const char *s)
{
    size_t len = strlen(s);
    char *copy = malloc(len + 1);    // +1 – место под '\0'
    if (copy == NULL)
        return NULL;
    memcpy(copy, s, len + 1);        // копируем и ноль тоже
    return copy;                     // владелец – вызывающий
}
```

## Главное

Без единицы блок короче строки ровно на байт, и ноль уходит за его границу. Без санитайзера такая программа обычно работает: `malloc` выдаёт блоки с запасом. Строка — такой же блок в куче: у него один владелец, и `free` вызывается ровно один раз.

## Во что обходится ноль на конце

Длина строки не хранится нигде, поэтому каждый `strlen` — это проход по всем байтам. Два цикла по строке из 200 000 символов:

```
for (size_t i = 0; i < strlen(s); i++) ... // 306.719 мс
size_t len = strlen(s);
for (size_t i = 0; i < len; i++) ... // 0.180 мс
```

Разница в **1700** раз. В первом случае цикл стоит  $O(n^2)$  вместо  $O(n)$ .

### Как это выглядит в Python

`len(s)` в Python стоит фиксированное время при любой длине: длина лежит в заголовке объекта, её не надо вычислять. Место, где она лежит, будет показано дальше.

РАЗДЕЛ 2

# Структуры

---

## Несколько значений под одним именем

```
struct Person {  
    char *name;      // указатель на строку в куче  
    int age;  
    double score;  
};  
  
struct Person p = {NULL, 17, 4.5};  
p.age = 18;           // поле через точку  
struct Person *q = &p;  
q->age = 19;          // через указатель – стрелка
```

Структура — это несколько переменных, лежащих в памяти подряд и названных одним именем. Поля могут быть любых типов, включая массивы, указатели и другие структуры.

Запись `q->age` — сокращение для `(*q).age`: точка связывает сильнее звёздочки, и стрелка появилась из-за неудобства скобок.

## Структура копируется целиком

```
struct Point { int x; int y; int trail[4]; };  
  
struct Point a = {1, 2, {10, 20, 30, 40}};  
struct Point b = a;           // копируются все 24 байта  
a.x = 777;                     // b.x попрежнему 1
```

Вчера `b = a` для массивов не компилировалось вовсе. Для структур присваивание законно и копирует все байты, включая массив `trail`: массив внутри структуры — часть её значения.

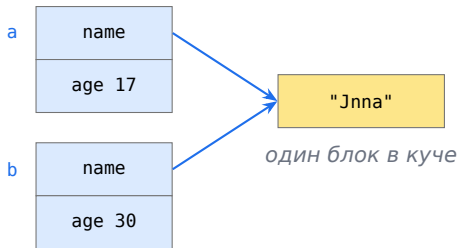
### Аргумент тоже копия

Структура передаётся в функцию по значению. Функция получает копию и меняет копию; чтобы изменения были видны снаружи, передают `struct Point *`. Заодно это дешевле: указатель — 8 байт при любом размере структуры.

## Что копируется, когда внутри указатель

```
struct Person a;  
a.name = my_strdup("Anna");  
a.age = 17;
```

```
struct Person b = a;  
b.age = 30;           // своё  
b.name[0] = 'J';      // общее
```



Скопировался указатель, а не то, на что он показывает. Поле `age` у копии своё, строка — общая, и `free` для неё вызывается один раз.

### Как это выглядит в Python

Так же различаются `copy.copy` и `copy.deepcopy`. Глубокая копия в C пишется руками: новый блок под строку и `my_strdup` в него.

# Массив структур в куче

```
struct Person *people = malloc(n * sizeof *people);
for (size_t i = 0; i < n; i++)
    people[i].name = my_strdup(word);    // блок на каждое имя

for (size_t i = 0; i < n; i++)
    free(people[i].name);                // сначала строки
free(people);                            // потом массив
```

Блоков в куче здесь `n + 1`: один под массив структур и по одному под каждое имя.

## Главное

Порядок освобождения — обратный порядку выделения. После `free(people)` читать `people[i].name` уже нельзя, а без этого адреса строку не освободить: получится `heap-use-after-free` или утечка.

РАЗДЕЛ 3

# Выравнивание

---

## Почему адрес не любой

Процессор читает память словами, а не байтами, и для каждого типа есть требование: адрес должен делиться на его размер. Это требование называется выравниванием и запрашивается оператором `_Alignof`.

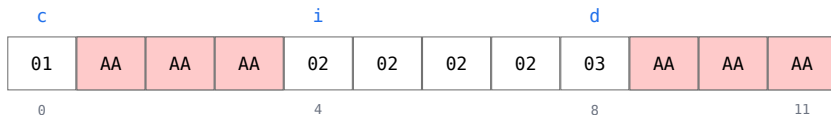
| тип                             | sizeof | выравнивание |
|---------------------------------|--------|--------------|
| char                            | 1      | 1            |
| int                             | 4      | 4            |
| double, size_t, любой указатель | 8      | 8            |

Компилятор обязан разместить каждое поле структуры по подходящему адресу. Единственный способ это сделать — вставлять между полями неиспользуемые байты.

`malloc` возвращает адрес, подходящий для любого типа: блок из кучи выровнен по 16 байтам, и положить в него можно что угодно.

# Дыры внутри структуры

```
struct Mixed { char c; int i; char d; };
```



Сумма размеров полей — 6 байт, `sizeof(struct Mixed)` — 12. Красным помечены байты, которых не касалось ни одно присваивание: три перед `i`, чтобы его смещение делилось на 4, и три в конце.

Хвостовые байты нужны массиву: в `struct Mixed arr[2]` второй элемент обязан начинаться с адреса, кратного своему выравниванию, следовательно `sizeof(struct Mixed)` делится на выравнивание себя.

## Как считается размер

---

Правило выравнивания структуры и его полей просто:

1. Поля размещаются в порядке объявления, каждое — по ближайшему подходящему адресу.
2. Выравнивание структуры равно наибольшему из выравниваний её полей.
3. Размер округляется вверх до кратного этому числу.

```
#include <stddef.h>
printf("%zu\n", offsetof(struct Mixed, i)); // 4
```

`offsetof` даёт смещение поля от начала структуры. Проверять себя им полезно, особенно для больших структур.

Единственное, что стандарт запрещает компилятору, — переставлять поля местами. Порядок в памяти совпадает с порядком в объявлении всегда.

## Порядок полей важен

```
struct Bad {  
    char    flag;  
    double  value;  
    int     count;  
};    // 24 байта
```

```
struct Good {  
    double  value;  
    int     count;  
    char    flag;  
};    // 16 байт
```

Поля одни и те же, сумма их размеров — 13 байт. Дыры занимают 11 байт в первой структуре и 3 во второй; на миллионе записей это 22.9 МБ против 15.3 МБ.

### Правило порядка полей

Объявляйте поля от больших к меньшим: дыры остаются только в конце, и их не больше, чем `_Alignof(struct ...) - 1`. Упаковать структуру до 13 байт умеет расширение `__attribute__((packed))` — ценой того, что обращение к невыровненному полю обходится дороже.

РАЗДЕЛ 4

## **Ответ на вопрос дня**

---

# Объект в CPython

---

Каждое значение в Python — блок в куче, и начинается он с двух полей, одинаковых у всех типов:

```
typedef struct {  
    Py_ssize_t ob_refcnt;    // счётчик ссылок, 8 байт  
    PyObject *ob_type;      // указатель на тип, 8 байт  
} PyObject;
```

Шестнадцать байт до начала собственных данных объекта:

`sys.getsizeof(None)` равен 16, потому что своих данных у `None` нет.

`ob_type` отвечает на вопрос «что это за объект»: по нему находятся все операции. Отсюда динамическая типизация — тип хранится в значении, а настоящий тип объекта один — `PyObject`.

`ob_refcnt` считает, сколько имён ссылаются на объект. Когда счётчик доходит до нуля — интерпритатор вызывает `free`.

## Целое число

```
struct PyLong {  
    size_t    refcnt;        // 8  
    void      *type;         // 8  
    uintptr_t tag;           // 8 — число цифр и знак  
    uint32_t  digit[1];      // 4 — цифры по 30 бит  
};
```

Заголовок занимает 24 байта, одна цифра — ещё 4. Отсюда 28.

|                | 1  | $2^{*30}$ | $2^{*60}$ | $2^{*90}$ |
|----------------|----|-----------|-----------|-----------|
| цифр по 30 бит | 1  | 2         | 3         | 4         |
| sizeof         | 28 | 32        | 36        | 40        |

Каждая следующая цифра добавляет ровно 4 байта. Так выглядит ответ на вопрос первого дня: длинная арифметика — это массив цифр переменной длины, лежащий в том же блоке, что и заголовок.

## Строка

|                        | <code>" "</code> | <code>"a"</code> | <code>"ab"</code> | <code>"hello"</code> | <code>"ёж"</code> | <code>"привет"</code> |
|------------------------|------------------|------------------|-------------------|----------------------|-------------------|-----------------------|
| <code>getsizeof</code> | 41               | 42               | 43                | 46                   | 62                | 70                    |

Пока все коды меньше 128, на символ уходит по байту: 40 байт заголовка, длина строки и байт под завершающий ноль — сишная строка внутри питоновской, чтобы её можно было отдать библиотечной функции без преобразования. Буква с большим кодом делает символ двухбайтовым, а заголовок — 56-байтовым.

### Главное

В заголовке лежат, кроме счётчика ссылок и типа, длина и хеш. Длина хранится, поэтому `len(s)` стоит фиксированное время при любом размере строки. Хеш вычисляется при первом обращении и сохраняется рядом — на этом будет построен завтрашний день.

## Почему строки неизменяемы

Хранимые в заголовке длина и хеш верны только до тех пор, пока байты строки не меняются. Из запрета на изменение следуют три вещи сразу:

- **Хеш считается один раз.** Ключ словаря можно хешировать только при вставке — завтра на этом держится весь день.
- **Одинаковые строки это один литерал.** Литералы в исходнике существуют в одном экземпляре: `a = "hello"; b = "hello"` даёт `a is b`. То же делает C для своих литералов.
- **Срез и присваивание не копируют данные.** Копируется указатель, счётчик ссылок увеличивается на единицу.

### Главное

Цена запрета — каждое изменение создаёт новый объект. Строка на миллион символов, у которой надо поменять один байт, копируется целиком.

## Чем за это платят

```
s = ""  
for i in range(n):  
    s += "x"
```

| n                                   | 25 000 | 50 000  | 100 000 |
|-------------------------------------|--------|---------|---------|
| обычный цикл                        | 0.7 мс | 1.3 мс  | 2.6 мс  |
| со второй ссылкой на <code>s</code> | 5.1 мс | 20.9 мс | 84.2 мс |

Каждый шаг обязан создать новую строку и скопировать в неё старую, то есть цикл стоит  $O(n^2)$ . Но первая строка таблицы показывает другое: время растёт линейно.

Причина — в счётчике ссылок: если на строку ссылается ровно одно имя и оно же стоит слева от `+=`, интерпретатор расширяет блок на месте. Достаточно сохранить промежуточный результат вторым именем, и оптимизация включается — вторая строка учетверяется при удвоении `n`.

Надёжный способ собрать строку — `"".join(parts)`: каждый байт копируется один раз.

### Почему `sizeof(1)` равно 28

Значение в Python — блок в куче, начинающийся с заголовка: счётчик ссылок, указатель на тип, а у числа ещё и тег с числом цифр. Это 24 байта. Само число занимает одну тридцатибитную цифру, то есть 4 байта. Каждая следующая цифра добавляет ещё 4 — отсюда и произвольная длина целых, и 28 у единицы.

### Почему строку нельзя изменить

В заголовке строки лежат её длина и хеш. Оба верны, пока байты не меняются, и на этом держатся быстрый `len`, словарь с ключами-строками и общие литералы. Изменение потребовало бы пересчитать хеш у всех, кто уже сослался на строку, поэтому вместо изменения создаётся новый объект.

## Итог дня

---

- Строка в C — массив `char` с нулевым байтом на конце. Длина не хранится, поэтому `strlen` стоит  $O(n)$ , а место под ноль просится у `malloc` отдельной единицей.
- Литерал лежит в `.rodata`, области только для чтения, и существует в одном экземпляре. `char s[]` делает копию, `const char *s` — нет.
- `==` для строк сравнивает адреса, содержимое сравнивает `strcmp`. Размещения строковых функций не знают.
- Структура копируется целиком, включая массивы внутри. Указатель внутри копируется как указатель: блок остаётся общим.
- Между полями стоят дыры: поле выровнено по своему размеру, размер структуры кратен её выравниванию. Порядок от больших к меньшим убирает почти все дыры.
- Объект в Python — блок с заголовком: счётчик ссылок, тип, длина, хеш. Неизменяемость строк — условие того, что хэш сохраняется.

**Задачи дня:** `day4-strings-structs/tasks/`

Вопросы к защите открыты: `defense/questions.md` — посмотрите их до того, как начнёте решать.